

Mastering Reinforcement Learning Basics & MDPs

Complete Beginner-Friendly Breakdown: Continuous Markov Decision Processes, Bellman Self-Consistency, Algorithm Anatomy & Bi-Manual Slicing

Instructor: Prof. Sergey Levine (UC Berkeley)

Companion: DEX-ROB Lab, Tianjin University

Frameworks: Achiam (Spinning Up) + Levine Formalism

0. The "Mental Map": Why Does Lecture 4 Exist?

In previous lectures, you learned about **Imitation Learning** (Behavioral Cloning). Imitation learning is simple: you record a human operating the robot and train a neural network to mimic the human. But what if you *don't* have 200 hours of perfect human demonstrations? Or what if a human cannot accurately control high-frequency (1 kHz) knife impedance forces during tomato skin rupture?

This is why Reinforcement Learning (RL) exists. Instead of copying a human, the robot discovers how to cut tomatoes on its own through simulated trial-and-error. Lecture 4 establishes the fundamental mathematical and conceptual language of RL: *Markov Decision Processes (MDPs)*, *Cumulative Objectives*, *Value Functions*, and *the Algorithm Taxonomy*.

PART 1: MDPs

States, Actions,
Markov Property

PART 2: Objective

Long-Term Returns,
Discount Factor γ

PART 3: Anatomy

Sample $\rightarrow$ Evaluate
 $\rightarrow$ Improve Loop

PART 4: Values

$V(s)$ & $Q(s,a)$
Advantage Function

PART 5: Taxonomy

PPO vs SAC vs
Model-Based

Part 1: The Markov Decision Process (MDP) De-Mystified

(Slides 1–9) At its core, RL models the interaction between an intelligent decision-maker (the robot's neural network) and the surrounding physical universe.

1.1 From Markov Chain to Markov Decision Process

A **Markov Chain** is just a sequence of random events where no one has any control (e.g., the weather transitions from sunny to rainy). A **Markov Decision Process (MDP)** adds an active agent that takes actions to alter the future and receives rewards based on performance.

MDP Tuple: $\mathbf{M} = \{ \mathbf{S}, \mathbf{A}, \mathbf{T}, \mathbf{R}, \gamma \}$

Element	Formal Math	Plain-English Meaning in Your Tomato Cutting Paper
State (S)	$\mathbf{s}_t \in \mathcal{S}$	The state vector (33 numbers): robot joint angles, knife 3D position & velocity, TacBlade 6-axis contact forces, and acoustic vibration.
Action (A)	$\mathbf{a}_t \in \mathcal{A}$	The continuous motor commands (6 numbers): downward feed rate delta, lateral sawing velocity, and vertical stiffness/damping adjustments ($\Delta K, \Delta D$).
Transitions (T)	$p(\mathbf{s}_{t+1} \mathbf{s}_t, \mathbf{a}_t)$	The Physics Simulator (Isaac Sim / PhysX 5) . It calculates how the tomato deforms and where the knife moves when you apply action $\mathbf{a}_t$.
Reward (R)	$r(\mathbf{s}_t, \mathbf{a}_t)$	The scoring formula: +points for slicing deeper and sawing, - penalties for squishing force ($>8\text{N}$).
Discount (γ)	$\gamma \in [0, 1)$	Set to 0.99 in SkRL: tells the robot to care about finishing the complete cut rather than just touching the surface.

1.2 The "Goldfish Memory" Rule: The Markov Property

INTUITION: WHAT IS THE MARKOV PROPERTY?

A system is **Markovian** if the future depends *only* on what is happening right now (state $\mathbf{s}_t$ and action $\mathbf{a}_t$), and **NOT** on the entire past history. The system has "goldfish memory"—it doesn't care what happened 10 seconds ago.

DIRECT ROBOTICS IMPLICATION FOR PAPER 1

Why did we design $\mathbf{s}_t \in \mathbb{R}^{33}$? If you only gave the neural network the knife's *position* (z), the system would **violate the Markov property!** Why? Because seeing a blade at height 5 cm tells you nothing about whether the knife is currently speeding downward at 50 mm/s or lifting upward. By adding **velocities** ($\mathbf{v}_{\text{knife}}$) and **rates of force change** into the state, step $\mathbf{s}_t$ holds all information needed to predict step $\mathbf{s}_{t+1}$.

Part 2: Defining the RL Objective & The Markov Chain View

(Slides 10–16) What does it mean for an algorithm to "learn"? It means tuning the neural network weights θ to maximize total expected reward.

2.1 The Objective: Why Immediate Greedy Actions Fail

$$\theta^* = \arg\max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

In Slide 11, Levine shows the car crash: if you drive at 100 km/h toward another car, slamming the brakes 1 meter away is the "best" immediate action, but you will still crash! The true mistake was made 5 seconds earlier.

THE TOMATO SKIN FRACTURE ANALOGY

When cutting a ripe tomato, the knife presses against tough elastic skin. If the robot acts greedily to maximize downward motion, it pushes down with 15 Newtons. At $t = 50$, the skin suddenly pops (rupture fracture). Because downward resistance vanishes in under 5 milliseconds, the knife violently slams into the cutting board and explodes the tomato.

RL solves this: It trains the policy to start sawing laterally and increasing damping (ΔD) *before* the fracture point, because RL optimizes for the **total cut quality**, not just the current millisecond!

2.2 The Markov Chain View: Unpacking Slide 12 & 13

In Slide 12, Levine asks: *"Is there a simpler way to write this?"* Instead of treating the entire 500-step trajectory τ as one giant joint probability distribution, we use the **linearity of expectation**:

$$\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] = \sum_{t=1}^H \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim p_{\theta}(\mathbf{s}_t, \mathbf{a}_t)} \left[r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Plain English: *"The average total score of the cut equals the sum of the average scores at each individual second."* We can bundle state and action into a single unit $(\mathbf{s}_t, \mathbf{a}_t)$ and step forward like links in a chain.



Part 3: The Anatomy of an RL Algorithm

(Slides 17–21) Every modern RL algorithm in existence—from simple DQN to the advanced PPO and SAC you run in SkRL—is made of the exact same 3-step loop:



3.1 Which Part is the Bottleneck? (Slide 21)

- **On a Real Physical Robot:** Step 1 is the painful bottleneck! You have to physically wait in real time (1x). If a tomato cut takes 10 seconds, collecting 100,000 cuts takes **11.5 days** of continuous robot wear-and-tear!
- **In Isaac Lab (Your Setup):** We solve this bottleneck by running **1,024 to 4,096 parallel robot arms simultaneously on your RTX GPU**. Physics runs at **1,000x to 10,000x real time**, allowing your robot to experience 10 years of cutting practice in just 45 minutes!

Part 4: Value Functions, Q-Functions & Bellman Self-Consistency

(Slides 22–26 & Spinning Up ch07) In **Spinning Up in Deep RL**, Joshua Achiam emphasizes that value functions are central because they obey **Bellman Self-Consistency**: the value of your starting point equals the immediate reward plus the value of wherever you land.

4.1 The 4 Core Value Functions

Function	Mathematical Definition	Physical Role in Dual-Arm Slicing
On-Policy State Value $V^\pi(s)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$	Expected total cut score if the knife is currently at state s and continues following policy π .
On-Policy Action Value $Q^\pi(s, a)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$	Expected score if the knife takes specific action a right now, and then continues following policy π .
Optimal State Value $V^*(s)$	$\max_{\pi} V^\pi(s) = \max_a Q^*(s, a)$	The absolute highest achievable cut quality possible from state s under the perfect policy.
Optimal Action Value $Q^*(s, a)$	$\max_{\pi} Q^\pi(s, a)$	The highest achievable score starting from state s , taking action a , and acting optimally thereafter.

4.2 The Bellman Equations

Bellman Expectation Equation for V : $V^\pi(s) = \mathbb{E}_{a \sim \pi, s' \sim P} \left[r(s, a) + \gamma V^\pi(s') \right]$

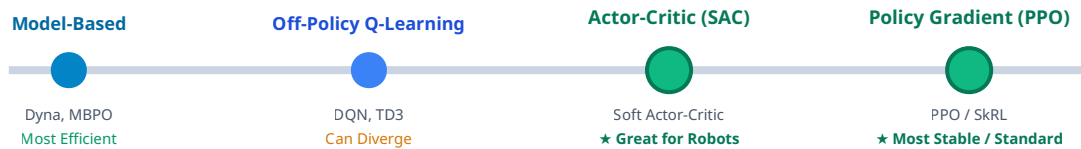
Bellman Optimality Equation for Q : $Q^*(s, a) = \mathbb{E}_{s' \sim P} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right]$

SPINNING UP BUG ALERT: THE $[N]$ VS $[N, 1]$ TENSOR BROADCASTING TRAP

Joshua Achiam highlights this as the #1 silent bug in RL implementations: If `rewards` has shape `[N]` (1D) and `next_values` has shape `[N, 1]` (2D), computing `targets = rewards + gamma * next_values` in PyTorch does NOT raise an error! Instead, Python broadcasts the two shapes into an **$[N, N]$ matrix**! The Critic trains on random pairwise broadcasted values, loss stays finite, but the policy never learns. Always ensure shapes are strictly aligned using `.view(-1, 1)` or `.squeeze()`.

Part 5: Taxonomy of RL Algorithms — Which One for Your Paper?

(Slides 27–41) Levine organizes all model-free algorithms along two fundamental trade-offs: **Sample Efficiency** and **Stability**.



5.1 The Two Fundamental Trade-Offs

Dimension	Sample Efficiency	Stability & Convergence
On-Policy (PPO)	Lowest: Discards data after each batch. Needs millions of simulator steps.	Highest: True gradient ascent on clipped objective. Extremely forgiving to tune. (<i>Isaac Lab compensates via 4,096 parallel GPU envs!</i>)
Off-Policy (SAC)	High: Re-uses old trajectories from a replay buffer over and over.	Moderate: Needs careful tuning of entropy temperature α and target network updates to avoid overestimating Q-values.
Pure Q-Learning (DQN)	Moderate	Unstable on Continuous Arms: Computing argmax over continuous actions (torques) is mathematically intractable.
Model-Based RL	Highest: Tries to learn a neural network predicting squashy tomato physics.	Brittle: Compounding model simulation errors lead to physical exploitation and catastrophic real-world failure.

Part 6: Interactive Tablet Self-Test Quiz (Test Your Understanding)

Question 1: If an environment state consists only of the knife's 3D Cartesian position $[x, y, z]$, does this satisfy the Markov property for cutting soft fruit?

Answer: No. A static position does not tell the robot whether the knife is accelerating downward or moving laterally, nor does it capture the accumulated elastic strain inside the tomato cuticle. To satisfy the Markov property, the state must include knife velocity ($\dot{z}$), lateral sawing speed, and TacBlade force derivatives ($\dot{F}_z$).

Question 2: In Bellman equations, what is the conceptual difference between $V^\pi(s)$ and $Q^\pi(s, a)$?

Answer: $V^\pi(s)$ is the expected return from state s when following policy π immediately. $Q^\pi(s, a)$ evaluates taking a specific designated action a first, and only following policy π on subsequent steps. The difference $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ tells us whether action a was superior to the policy's average choice.

Question 3: Why does Isaac Lab prefer on-policy PPO over off-policy SAC, despite PPO being less sample-efficient?

Answer: On GPU simulators like Isaac Lab, sample generation is virtually free (generating 260,000 steps in 50 ms across 4,096 parallel environments). PPO's stability, lack of memory-heavy replay buffers, and clipped objective allow straightforward scaling to massive parallel streams, whereas off-policy SAC causes GPU memory bottlenecks when managing replay buffers for 4,096 concurrent environments.

Part 7: Thesis Defense Master Cheatsheet (Lecture 4 Focus)

Q1: "WHY IS YOUR TOMATO SLICING TASK FORMULATED AS AN MDP RATHER THAN A POMDP?"

Answer: "While internal fruit tissue state is not directly visible, our multi-modal sensor fusion—combining TacBlade 6-axis forces, tactile contact patch distribution, acoustic emission, and end-effector kinematics—provides a sufficiently rich 33-dimensional state representation such that the Markov property is preserved for continuous compliant control."

Q2: "WHAT IS THE PHYSICAL INTERPRETATION OF SETTING DISCOUNT FACTOR $\gamma = 0.99$?"

Answer: "At 60 Hz control frequency, an effective time horizon is $1 / (1 - \gamma) \approx 100$ steps (approx. 1.6 seconds). This ensures the robot policy plans ahead for the complete skin rupture and tissue sawing cycle, preventing short-sighted greedy downward thrusts."

Q3: "HOW DOES THE ADVANTAGE FUNCTION $A(S, A)$ PROTECT THE TOMATO FROM VIOLENT MOTOR COMMANDS?"

Answer: "In delicate pre-puncture states ($V(s) \approx +50$), an action that applies excessive downward pressure causes bruising and fluid leakage, yielding $Q(s, a) \approx -30$. The resulting negative advantage ($A = -80$) suppresses this violent command in the policy gradient update, steering the policy toward compliant lateral sawing."